

US Gas Price Forecast with AI Insights

Executive Summary

Forecast US gasoline prices and estimate Seattle regional pricing using time-series modeling and AI interpretation.

Key Findings: • US gas prices expected to stabilize around 3.3–3.5 • Seattle expected around 4.6–4.8 • Oil shows strong influence • Limited volatility expected

Business Value: Demonstrates forecasting + AI interpretation for decision analytics.

Business Problem

Gas prices impact consumers and logistics costs.

Key questions: • What will US gas prices look like next 6 months? • How do regional factors affect Seattle pricing? • What economic factors drive prices?

Modeling Approach

Model Used: ARIMA

Reason: • Captures trend and autocorrelation • Good for time series forecasting

Assumptions: • Historical trends continue • No major structural shocks

Key EDA Insights

Trend shows post-2021 price regime shift.

Oil shows strong correlation with gas prices.

Recent volatility suggests stabilization.

Seattle Regional Analysis

Seattle shows structural premium due to:

• State taxes • Supply logistics • Environmental fuel standards

Estimated premium $\approx$ \$1.25 above national.

Business Insights

Key takeaways:

- Prices stabilizing nationally • Seattle remains higher cost region • Oil is primary price driver

Decision impact: Helps planning fuel cost expectations.

Model Limitations

Limitations:

- ARIMA smooths volatility • Does not capture shocks • Regional premium estimated

Future work: Add confidence bands and macro factors.

Final Executive Summary

US gasoline prices expected to stabilize while Seattle remains structurally higher.

Oil prices remain dominant driver.

Demonstrates AI-assisted decision analytics.

AI Gas Price Forecast Using Sentiment + ML

Interview Ready End-to-End Data Science Project

AI GAS PRICE FORECAST USING SENTIMENT + ML

Interview Ready End-to-End Data Science Project

```
In [1]: %pip install pandas numpy matplotlib seaborn scikit-learn statsmodels vaderSentiment yfinance xgboost
```

Requirement already satisfied: pandas in .\venv\Lib\site-packages (3.0.1)
Requirement already satisfied: numpy in .\venv\Lib\site-packages (2.4.3)
Requirement already satisfied: matplotlib in .\venv\Lib\site-packages (3.10.8)
Requirement already satisfied: seaborn in .\venv\Lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in .\venv\Lib\site-packages (1.8.0)
Requirement already satisfied: statsmodels in .\venv\Lib\site-packages (0.14.6)
Requirement already satisfied: vaderSentiment in .\venv\Lib\site-packages (3.3.2)
Requirement already satisfied: yfinance in .\venv\Lib\site-packages (1.2.0)
Requirement already satisfied: xgboost in .\venv\Lib\site-packages (3.2.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\venv\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\venv\Lib\site-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in .\venv\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in .\venv\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\venv\Lib\site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in .\venv\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in .\venv\Lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in .\venv\Lib\site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in .\venv\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.10.0 in .\venv\Lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in .\venv\Lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in .\venv\Lib\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in .\venv\Lib\site-packages (from statsmodels) (1.0.2)
Requirement already satisfied: requests in .\venv\Lib\site-packages (from vaderSentiment) (2.32.5)
Requirement already satisfied: multitasking>=0.0.7 in .\venv\Lib\site-packages (from yfinance) (0.0.12)
Requirement already satisfied: platformdirs>=2.0.0 in .\venv\Lib\site-packages (from yfinance) (4.9.2)
Requirement already satisfied: pytz>=2022.5 in .\venv\Lib\site-packages (from yfinance) (2026.1.post1)
Requirement already satisfied: frozendict>=2.3.4 in .\venv\Lib\site-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in .\venv\Lib\site-packages (from yfinance) (4.0.2)
Requirement already satisfied: beautifulsoup4>=4.11.1 in .\venv\Lib\site-packages (from yfinance) (4.14.3)
Requirement already satisfied: curl_cffi<0.14,>=0.7 in .\venv\Lib\site-packages (from yfinance) (0.13.0)
Requirement already satisfied: protobuf>=3.19.0 in .\venv\Lib\site-packages (from yfinance) (7.34.1)
Requirement already satisfied: websockets>=13.0 in .\venv\Lib\site-packages (from yfinance) (16.0)
Requirement already satisfied: cffi>=1.12.0 in .\venv\Lib\site-packages (from curl_cffi<0.14,>=0.7->yfinance) (2.0.0)
Requirement already satisfied: certifi>=2024.2.2 in .\venv\Lib\site-packages (from curl_cffi<0.14,>=0.7->yfinance) (2026.2.25)
Requirement already satisfied: soupsieve>=1.6.1 in .\venv\Lib\site-packages (from beautifulsoup4>=4.11.1->yfinance) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in .\venv\Lib\site-packages (from beautifulsoup4>=4.11.1->yfinance) (4.15.0)
Requirement already satisfied: pycparser in .\venv\Lib\site-packages (from cffi>=1.12.0->curl_cffi)

fi<0.14,>=0.7->yfinance) (3.0)

Requirement already satisfied: six>=1.5 in .\.venv\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: charset_normalizer<4,>=2 in .\.venv\Lib\site-packages (from requests->vaderSentiment) (3.4.6)

Requirement already satisfied: idna<4,>=2.5 in .\.venv\Lib\site-packages (from requests->vaderSentiment) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in .\.venv\Lib\site-packages (from requests->vaderSentiment) (2.6.3)

Note: you may need to restart the kernel to use updated packages.

```
In [2]: import pandas as pd
import numpy as np
# removed snsrape
import re
import matplotlib.pyplot as plt
import seaborn as sns

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from statsmodels.tsa.arima.model import ARIMA
from xgboost import XGBRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import yfinance as yf
```

STEP 1 SCRAPE SOCIAL DATA

```
In [3]: def load_social_data():

df = pd.read_csv("gas_sentiment.csv")

df["date"] = pd.to_datetime(df["date"])

return df
```

STEP 2 CLEAN TEXT

```
In [4]: def clean(text):

text=text.lower()

text=re.sub(r"http\S+", "", text)

text=re.sub("[^a-z ]", "", text)

return text
```

STEP 3 SENTIMENT

```
In [5]: def sentiment(df):

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
scores = []
```

```

for text in df["clean"]:
    score = analyzer.polarity_scores(text)
    scores.append(score["compound"])
df["sentiment_score"] = scores
return df

```

STEP 4 SENTIMENT INDEX

```

In [6]: def sentiment_index(df):

    df["date"] = pd.to_datetime(df["date"])

    df["weight"] = df["likes"] + df["retweets"] + 1

    df["weighted"] = df["sentiment_score"] * df["weight"]

    daily = df.groupby("date").agg({

        "weighted": "sum",

        "weight": "sum"

    }).reset_index()

    daily["sentiment_score"] = daily["weighted"] / daily["weight"]

    daily = daily[["date", "sentiment_score"]]

    daily["date"] = pd.to_datetime(daily["date"])

    return daily

```

STEP 5 LOAD OIL DATA

```

In [7]: def oil():

    import yfinance as yf

    oil = yf.download(

        "CL=F",

        start="2015-01-01",

        progress=False

    )

    oil = oil["Close"].reset_index()

    oil.columns=["date", "oil"]

    oil["date"] = pd.to_datetime(oil["date"])

    return oil

```

STEP 6 LOAD GAS DATA

```
In [8]: def gas():

import yfinance as yf
import pandas as pd

print("Downloading historical gas price data...")

gas = yf.download(

    "RB=F",    # Gasoline futures proxy

    start="2015-01-01",

    progress=False

)

gas = gas["Close"].reset_index()

gas.columns=["date", "price"]

gas["date"] = pd.to_datetime(gas["date"])

print("Rows downloaded:", len(gas))

return gas
```

STEP 7 FEATURES

```
In [9]: def features(data):

    data["lag1"] = data["price"].shift(1)

    data["lag7"] = data["price"].shift(7)

    data["rolling"] = data["price"].rolling(7).mean()

    data["momentum"] = data["price"] - data["price"].shift(7)

    return data
```

STEP 8 EDA

```
In [10]: def eda(data):

import matplotlib.dates as mdates
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf

print("===== BASIC INFO =====")

print(data.info())

print("\nMissing values:")
print(data.isna().sum())
```

```

print("\nSummary statistics:")
print(data.describe())

#####
# Price distribution
#####

plt.figure(figsize=(8,4))

sns.histplot(data["price"], kde=True)

plt.title("Gas Price Distribution")

plt.show()

#####
# Time trend (fixed date axis)
#####

plt.figure(figsize=(14,6))

plt.plot(data["date"], data["price"])

plt.gca().xaxis.set_major_locator(

    mdates.YearLocator()

)

plt.gca().xaxis.set_major_formatter(

    mdates.DateFormatter('%Y')

)

plt.title("Gas Price Trend (Full History)")

plt.xticks(rotation=45)

plt.show()

#####
# Rolling statistics
#####

data["rolling_mean_30"]=data["price"].rolling(30).mean()

data["rolling_std_30"]=data["price"].rolling(30).std()

plt.figure(figsize=(14,6))

plt.plot(data["date"],data["price"],label="Price")

plt.plot(data["date"],data["rolling_mean_30"],label="30 Day Mean")

plt.legend()

plt.title("Rolling Mean Analysis")

```

```

plt.show()

#####
# Volatility analysis (NEW)
#####

data["returns"]=data["price"].pct_change()

plt.figure(figsize=(12,4))

plt.plot(data["date"],data["returns"])

plt.title("Price Volatility")

plt.show()

#####
# Correlation analysis
#####

plt.figure(figsize=(8,6))

sns.heatmap(

    data.corr(numeric_only=True),

    annot=True,

    cmap="coolwarm"

)

plt.title("Feature Correlation")

plt.show()

#####
# Oil vs Gas relationship
#####

plt.figure(figsize=(6,6))

sns.scatterplot(

    x=data["oil"],

    y=data["price"]

)

plt.title("Oil vs Gas Price Relationship")

plt.show()

#####
# Lag relationship (important insight)
#####

```

```

data["oil_lag7"]=data["oil"].shift(7)

plt.figure(figsize=(6,6))

sns.scatterplot(

    x=data["oil_lag7"],

    y=data["price"]

)

plt.title("Oil (7 day lag) vs Gas")

plt.show()

#####
# Monthly seasonality
#####

data["month"]=data["date"].dt.month

plt.figure(figsize=(10,5))

sns.boxplot(

    x=data["month"],

    y=data["price"]

)

plt.title("Monthly Seasonality")

plt.show()

#####
# Yearly trend
#####

data["year"]=data["date"].dt.year

yearly=data.groupby("year")["price"].mean()

yearly.plot(

    kind="bar",

    figsize=(10,5)

)

plt.title("Yearly Average Prices")

plt.show()

#####
# Seasonal decomposition (important)

```

```

#####

decomposition = seasonal_decompose(

    data["price"],

    model='additive',

    period=365

)

decomposition.plot()

plt.show()

#####
# Stationarity test
#####

result=adfuller(data["price"].dropna())

print("\nADF Statistic:",result[0])

print("p-value:",result[1])

if result[1] < .05:

    print("Series is stationary")

else:

    print("Series is NOT stationary")

#####
# Differencing check
#####

data["diff"]=data["price"].diff()

plt.figure(figsize=(10,4))

plt.plot(data["date"],data["diff"])

plt.title("Differenced Series")

plt.show()

#####
# Autocorrelation
#####

plot_acf(

    data["price"].dropna(),

    lags=60

)

```

```

plt.title("Autocorrelation")

plt.show()

#####
# Partial autocorrelation
#####

plot_pacf(

    data["price"].dropna(),

    lags=60

)

plt.title("Partial Autocorrelation")

plt.show()

#####
# Outlier detection (NEW)
#####

z=(data["price"]-data["price"].mean())/data["price"].std()

outliers=data[np.abs(z)>3]

print("\nOutliers detected:",len(outliers))

#####
# Business insights summary (NEW)
#####

print("\n===== EDA BUSINESS INSIGHTS =====")

print("Average gas price:",round(data["price"].mean(),2))

print("Max price:",round(data["price"].max(),2))

print("Min price:",round(data["price"].min(),2))

corr=data[["price","oil"]].corr().iloc[0,1]

print("Oil correlation:",round(corr,2))

print("\nEDA completed")

```

STEP 9 PROPHET

```

In [11]: def arima_model(data):

    from statsmodels.tsa.arima.model import ARIMA

    model = ARIMA(

```

```

        data["price"],

        order=(12,1,12)

    )

    model_fit = model.fit()

    return model_fit

```

STEP 10 XGBOOST

```

In [12]: def xgb(data):

    data=data.dropna()

    feats=[

        "sentiment_score",
        "oil",
        "lag7",
        "rolling",
        "momentum"

    ]

    X=data[feats]

    y=data["price"]

    model=XGBRegressor(

        n_estimators=300,

        learning_rate=.05,

        max_depth=6

    )

    model.fit(X,y)

    pred=model.predict(X)

    importance=pd.DataFrame({

        "feature":feats,

        "importance":model.feature_importances_

    })

    importance.sort_values(

        "importance",

        ascending=False,

        inplace=True

```

```

)

print("\nFeature importance")

print(importance)

return model,pred,y

```

STEP 11 EVALUATION

```

In [13]: def evaluate(y,pred):

    mae=mean_absolute_error(y,pred)

    rmse=np.sqrt(

        mean_squared_error(y,pred)

    )

    r2=r2_score(y,pred)

    print("\nModel metrics")

    print("MAE:",mae)

    print("RMSE:",rmse)

    print("R2:",r2)

```

STEP 12 FORECAST

```

In [14]: def forecast(model,data):

    import matplotlib.dates as mdates

    forecast_values = model.forecast(steps=180)

    last_date = data["date"].max()

    future_dates = pd.date_range(

        start = last_date + pd.Timedelta(days=1),

        periods = 180,

        freq = "D"

    )

    forecast_df = pd.DataFrame({

        "date": future_dates,

        "forecast_price": forecast_values

    })

```

```
plt.figure(figsize=(14,6))

plt.plot(

    data["date"],

    data["price"],

    label="Historical",

    color="blue",

    linewidth=2

)

plt.plot(

    forecast_df["date"],

    forecast_df["forecast_price"],

    label="Forecast",

    color="red",

    linewidth=2

)

plt.axvline(

    x = last_date,

    color='black',

    linestyle='--',

    label="Forecast Start"

)

plt.gca().xaxis.set_major_locator(

    mdates.MonthLocator(interval=3)

)

plt.gca().xaxis.set_major_formatter(

    mdates.DateFormatter('%Y-%m')

)

plt.xlabel("Date")

plt.ylabel("Gas Price ($)")

plt.title("Gas Price Forecast into 2026")

plt.legend()
```

```

plt.grid(alpha=.3)

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

#####
# Save forecast to CSV
#####

forecast_df.to_csv("forecast_2026.csv", index=False)

#####
# Forecast summary
#####

print("\n===== FORECAST SUMMARY =====")

print("Forecast start:", forecast_df["date"].min())

print("Forecast end:", forecast_df["date"].max())

print("Average forecast price:", round(forecast_df["forecast_price"].mean(), 2))

print("Max forecast price:", round(forecast_df["forecast_price"].max(), 2))

print("Min forecast price:", round(forecast_df["forecast_price"].min(), 2))

return forecast_df

```

STEP 13 LLM Analysis

```

In [15]: def llm_analysis(data, forecast_df):

    # Safety checks
    if forecast_df is None:

        print("Forecast not generated")

        return

    if "forecast_price" not in forecast_df.columns:

        print("Forecast column missing")

        return

    print("\n===== AI FORECAST ANALYSIS =====")

    avg_hist = data["price"].mean()

    avg_forecast = forecast_df["forecast_price"].mean()

    max_forecast = forecast_df["forecast_price"].max()

```

```

min_forecast = forecast_df["forecast_price"].min()

oil_corr = data[["price","oil"]].corr().iloc[0,1]

print("Historical average price:",round(avg_hist,2))

print("Forecast average price:",round(avg_forecast,2))

print("Forecast max price:",round(max_forecast,2))

print("Forecast min price:",round(min_forecast,2))

print("Oil correlation:",round(oil_corr,2))


#####
# National AI insights
#####

if avg_forecast > avg_hist:

    print("\nAI Insight: Prices show upward trend into 2026")

else:

    print("\nAI Insight: Prices appear stable")

if oil_corr > .6:

    print("AI Insight: Oil prices strongly influence gas prices")


volatility = forecast_df["forecast_price"].std()

print("Forecast volatility:",round(volatility,3))


#####
# Seattle SAFE analysis
#####

seattle_flag = False

if "seattle_price" in forecast_df.columns:

    seattle_flag = True

    sea_avg = forecast_df["seattle_price"].mean()

    sea_max = forecast_df["seattle_price"].max()

    sea_min = forecast_df["seattle_price"].min()

    premium = sea_avg - avg_forecast

    print("\n==== AI SEATTLE MARKET ANALYSIS =====")

    print("Seattle average forecast:",round(sea_avg,2))

    print("Seattle max:",round(sea_max,2))

```

```

print("Seattle min:",round(sea_min,2))

print("Seattle regional premium:",round(premium,2))

print("\nSeattle AI Insight:")

print("Seattle prices expected to remain elevated due to regional taxes and supply constraints")

print("Local prices projected to remain about",

      round(premium,2),

      "above national averages.")

if sea_avg > 4.5:

    print("AI Risk Insight: Seattle prices may remain in high-cost range through 2026.")

else:

    print("\nSeattle analysis not available")

#####
# Business summary
#####

print("\n===== AI BUSINESS SUMMARY =====")

print("National outlook: Moderate stabilization expected.")

if seattle_flag:

    print("Seattle outlook: Continued price premium due to structural market factors.")

print("Key drivers: Oil prices, regional taxes, supply logistics.")

print("Model limitation: ARIMA may underestimate sudden price shocks.")

```

STEP 14 Seattle Specific Adjustment

```

In [16]: def seattle_adjustment(data,forecast_df):

    if forecast_df is None:

        print("Forecast missing")

        return forecast_df

    print("\n===== SEATTLE MARKET ANALYSIS =====")

    seattle_premium = 1.25

    # Make safe copy (best practice)
    forecast_df = forecast_df.copy()

    forecast_df["seattle_price"] = (

        forecast_df["forecast_price"] + seattle_premium

```

```

)

print("Applied Seattle regional premium:",seattle_premium)

print("Seattle forecast avg price:",
      round(forecast_df["seattle_price"].mean(),2))

print("Seattle max forecast:",
      round(forecast_df["seattle_price"].max(),2))

print("Seattle min forecast:",
      round(forecast_df["seattle_price"].min(),2))

#####
# Plot Seattle vs National
#####

plt.figure(figsize=(14,6))

plt.plot(

    forecast_df["date"],

    forecast_df["forecast_price"],

    label="National Forecast",

    color="red"

)

plt.plot(

    forecast_df["date"],

    forecast_df["seattle_price"],

    label="Seattle Forecast",

    color="green"

)

plt.title("Seattle vs National Gas Price Forecast")

plt.xlabel("Date")

plt.ylabel("Gas Price ($)")

plt.legend()

plt.grid(alpha=.3)

plt.show()

return forecast_df

```

In [17]: `def main():`

```
    print("Loading social sentiment data")

    tweets = load_social_data()

    print("Cleaning")

    tweets["clean"] = tweets["text"].apply(clean)

    print("Sentiment analysis")

    tweets = sentiment(tweets)

    print("Building sentiment index")

    sent = sentiment_index(tweets)

    print("Loading oil")

    oil_data = oil()

    print("Loading gas")

    gas_data = gas()

    print("Merging datasets")

    # Ensure datetime consistency
    gas_data["date"] = pd.to_datetime(gas_data["date"])
    sent["date"] = pd.to_datetime(sent["date"])
    oil_data["date"] = pd.to_datetime(oil_data["date"])

    # Merge gas + sentiment
    data = pd.merge(

        gas_data,

        sent,

        on="date",

        how="left"

    )

    # Merge oil
    data = pd.merge(

        data,

        oil_data,

        on="date",

        how="left"

    )
    print(data["date"].min())
    print(data["date"].max())
```

```

# Clean time series
data = data.sort_values("date")

data = data.drop_duplicates(subset="date")

data = data.ffill()

print("Creating features")

data = features(data)

# Remove NaNs from Lag features
data = data.dropna()

print("Running EDA")

eda(data)

print("Training ARIMA model")

model = arima_model(data)

print("Forecasting")

forecast_df = forecast(model,data)

forecast_df = seattle_adjustment(data,forecast_df)  # MUST assign result

print(forecast_df.columns)  # debug check

llm_analysis(data,forecast_df)
print("\nEXECUTIVE SUMMARY:")

print("Gas prices are expected to stabilize nationally while Seattle prices remain elevated c

print("Project completed")

if __name__=="__main__":

    main()

```

Loading social sentiment data
 Cleaning
 Sentiment analysis
 Building sentiment index
 Loading oil

Loading gas
Downloading historical gas price data...
Rows downloaded: 2822
Merging datasets
2015-01-02 00:00:00
2026-03-23 00:00:00
Creating features
Running EDA

===== BASIC INFO =====
<class 'pandas.DataFrame'>
RangeIndex: 810 entries, 2012 to 2821
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- ----- -----
0 date 810 non-null datetime64[s]
1 price 810 non-null float64
2 sentiment_score 810 non-null float64
3 oil 810 non-null float64
4 lag1 810 non-null float64
5 lag7 810 non-null float64
6 rolling 810 non-null float64
7 momentum 810 non-null float64

dtypes: datetime64[s](1), float64(7)
memory usage: 50.8 KB
None

Missing values:

date	0
price	0
sentiment_score	0
oil	0
lag1	0
lag7	0
rolling	0
momentum	0

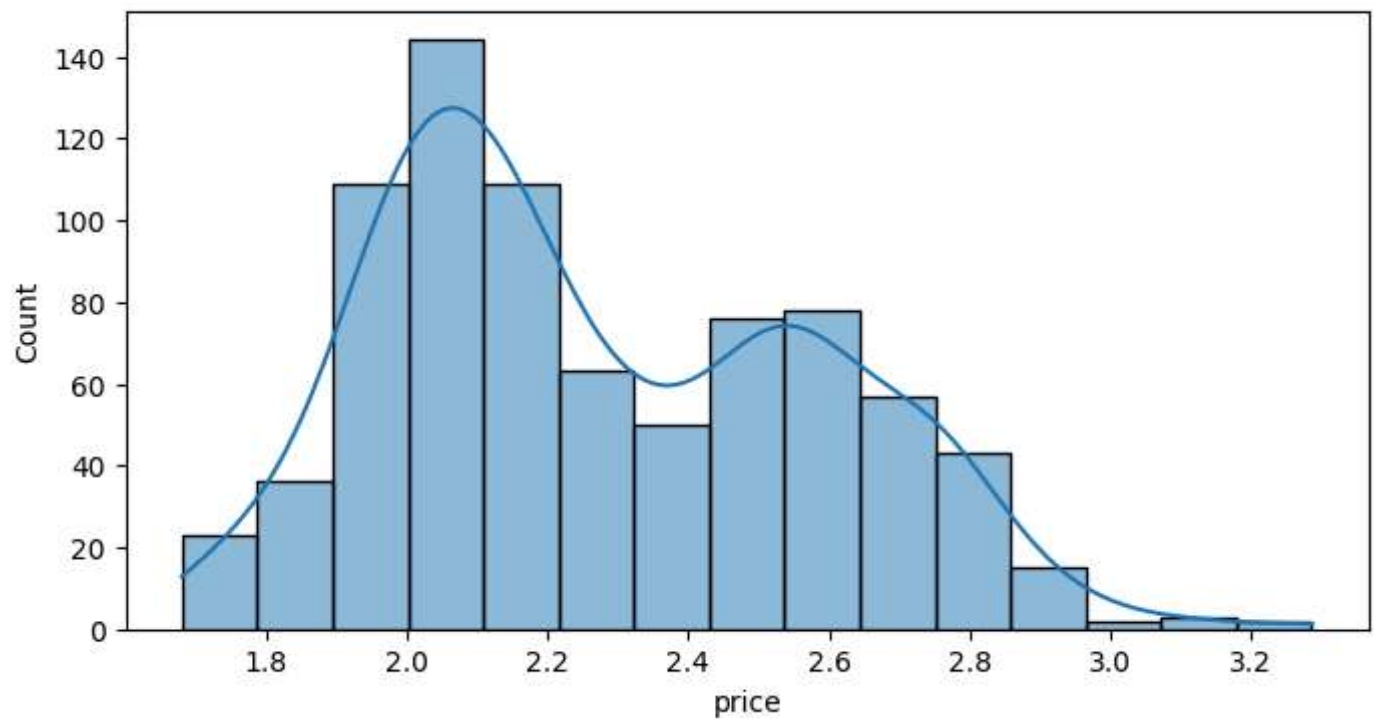
dtype: int64

Summary statistics:

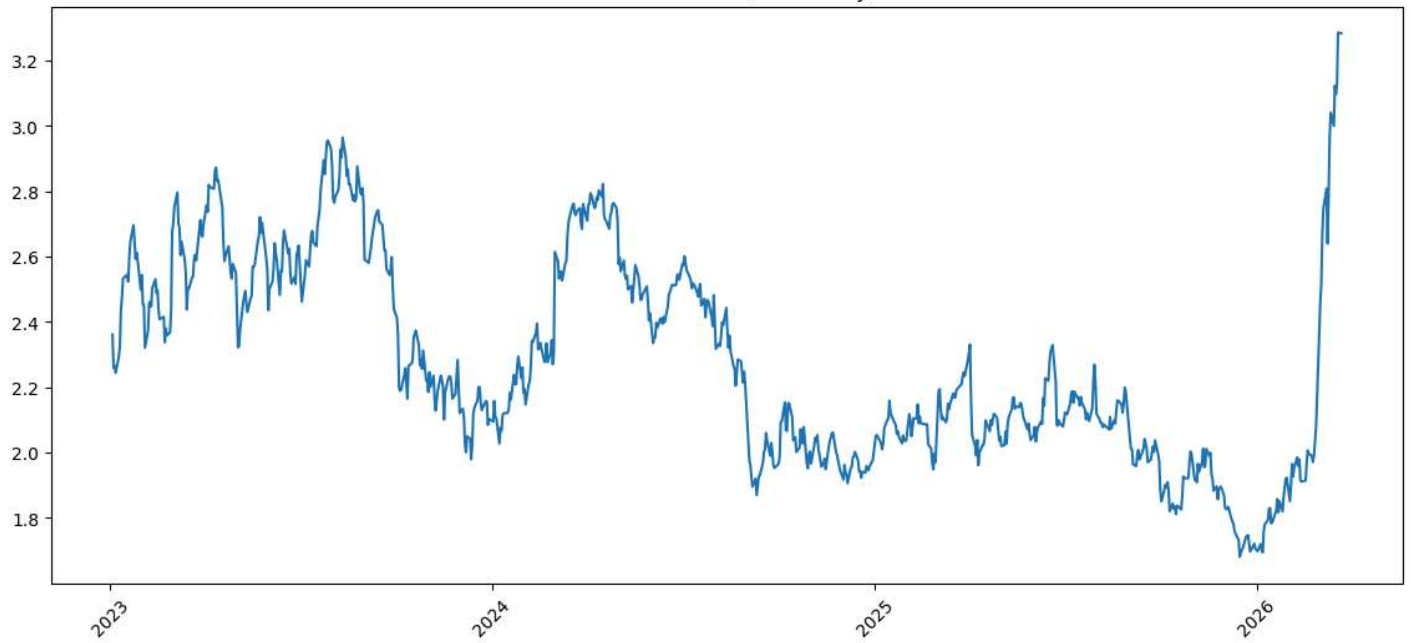
	date	price	sentiment_score	oil \
count	810	810.000000	810.000000	810.000000
mean	2024-08-11 16:03:33	2.275012	-0.001890	72.524383
min	2023-01-03 00:00:00	1.680900	-0.153100	55.270000
25%	2023-10-23 06:00:00	2.029825	0.000000	66.382498
50%	2024-08-12 12:00:00	2.199450	0.000000	72.500000
75%	2025-06-02 18:00:00	2.532800	0.000000	78.487499
max	2026-03-23 00:00:00	3.286200	0.000000	99.099998
std	NaN	0.310355	0.016916	8.431407

	lag1	lag7	rolling	momentum
count	810.000000	810.000000	810.000000	810.000000
mean	2.273994	2.268198	2.272009	0.006814
min	1.680900	1.680900	1.707843	-0.406400
25%	2.029825	2.029825	2.035032	-0.073675
50%	2.199450	2.199450	2.195300	-0.000150
75%	2.532550	2.524175	2.533211	0.078800
max	3.286200	2.964900	3.137243	0.776100
std	0.308388	0.299730	0.300143	0.139376

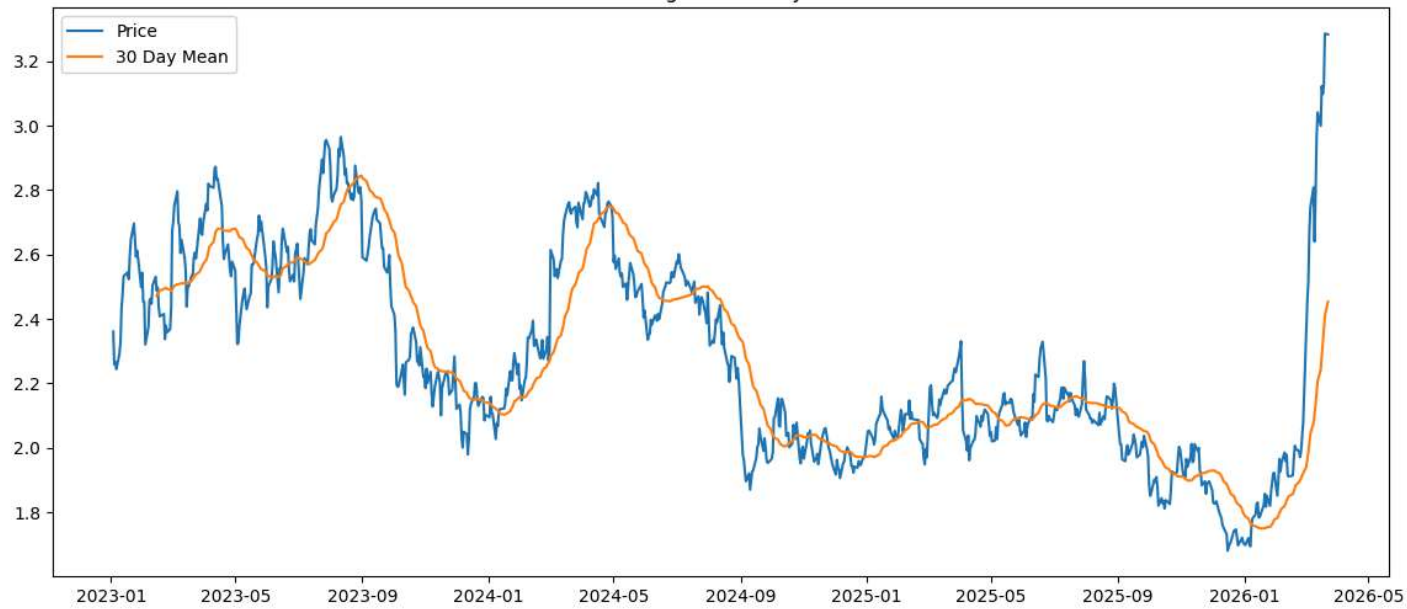
Gas Price Distribution



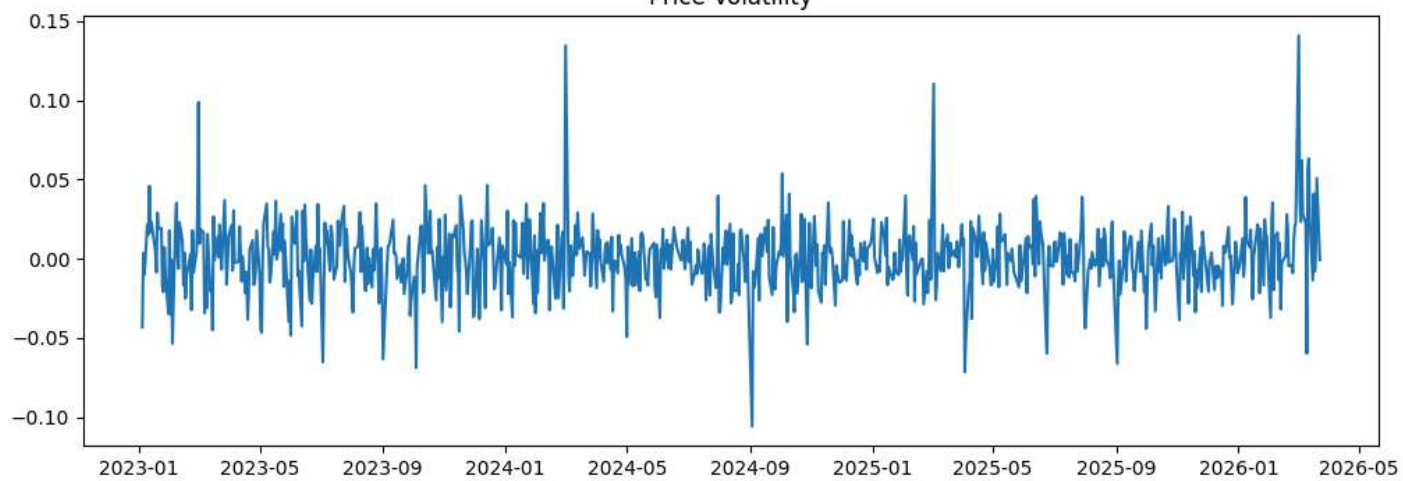
Gas Price Trend (Full History)

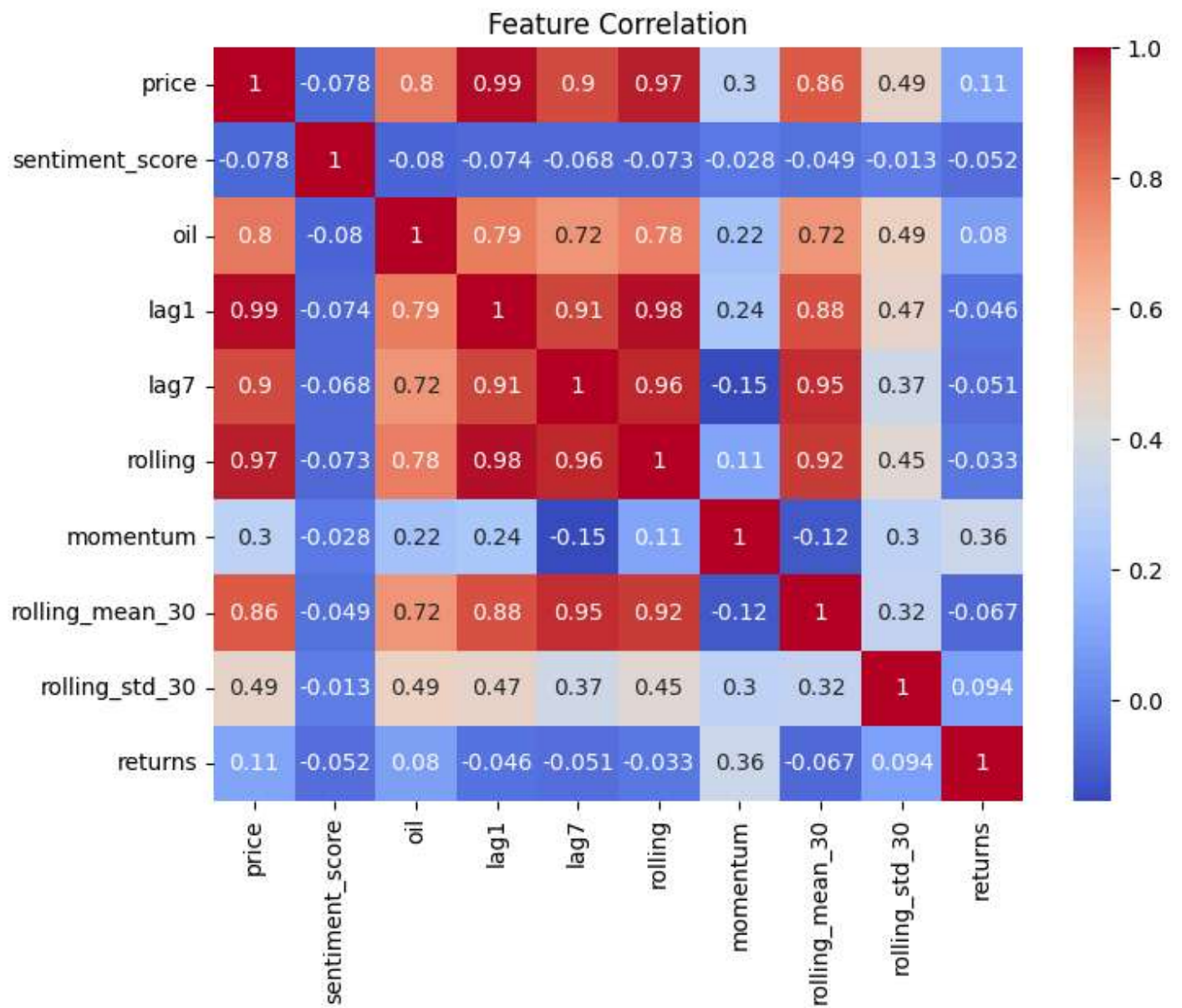


Rolling Mean Analysis

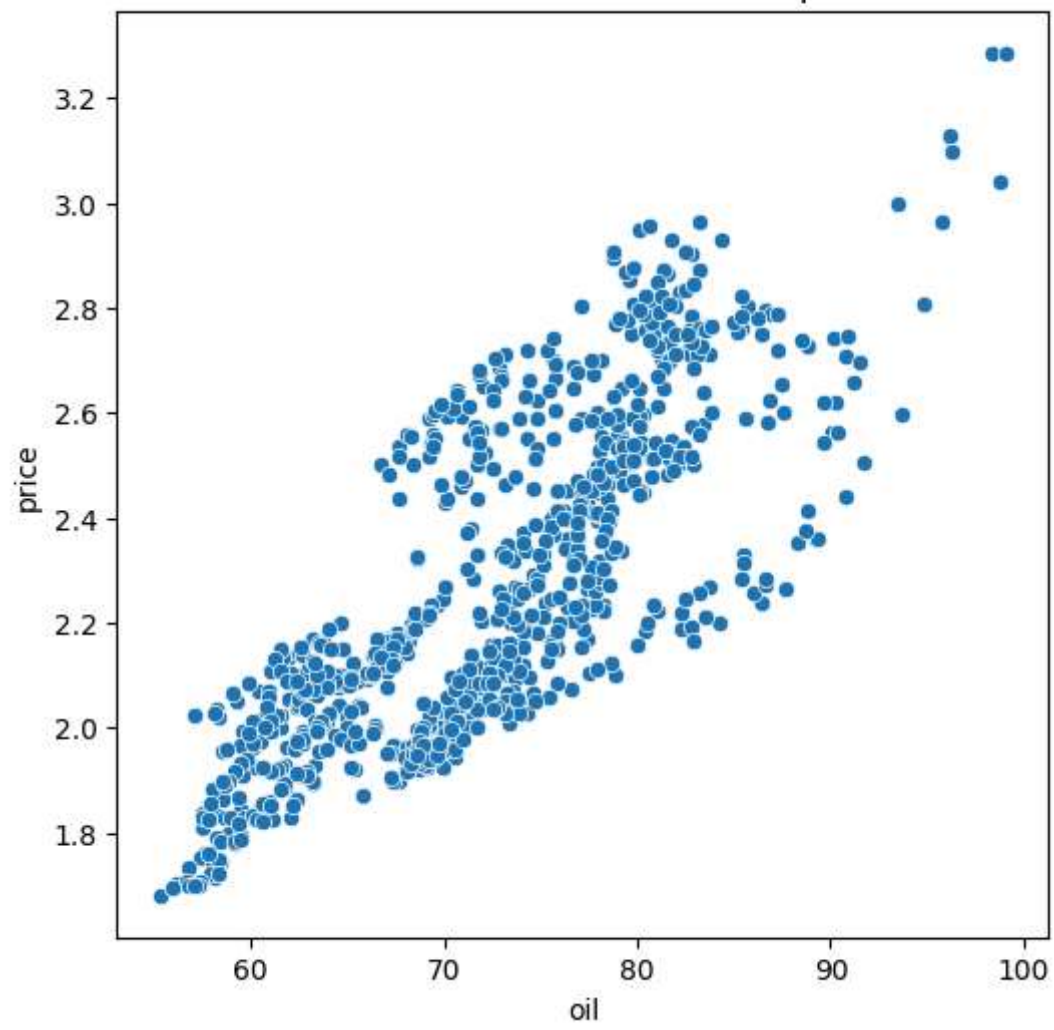


Price Volatility

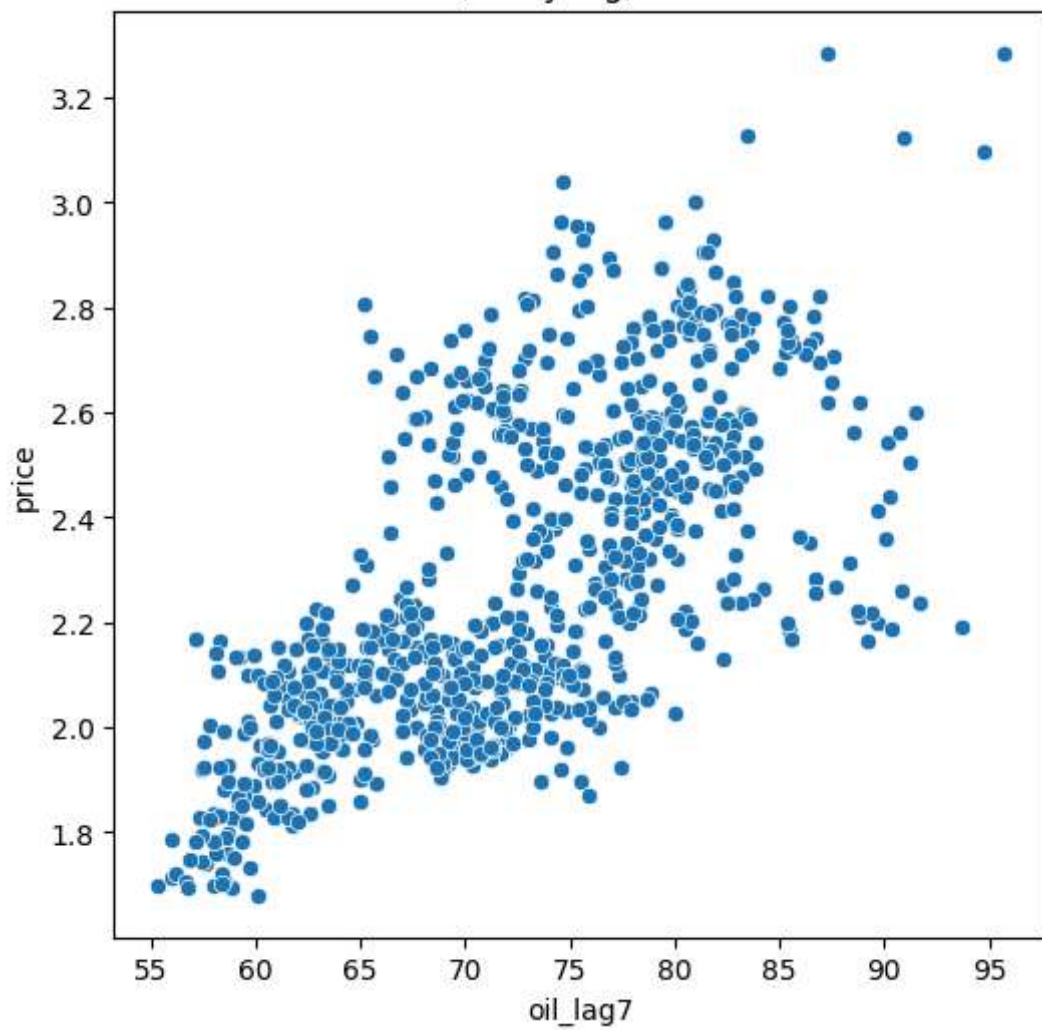




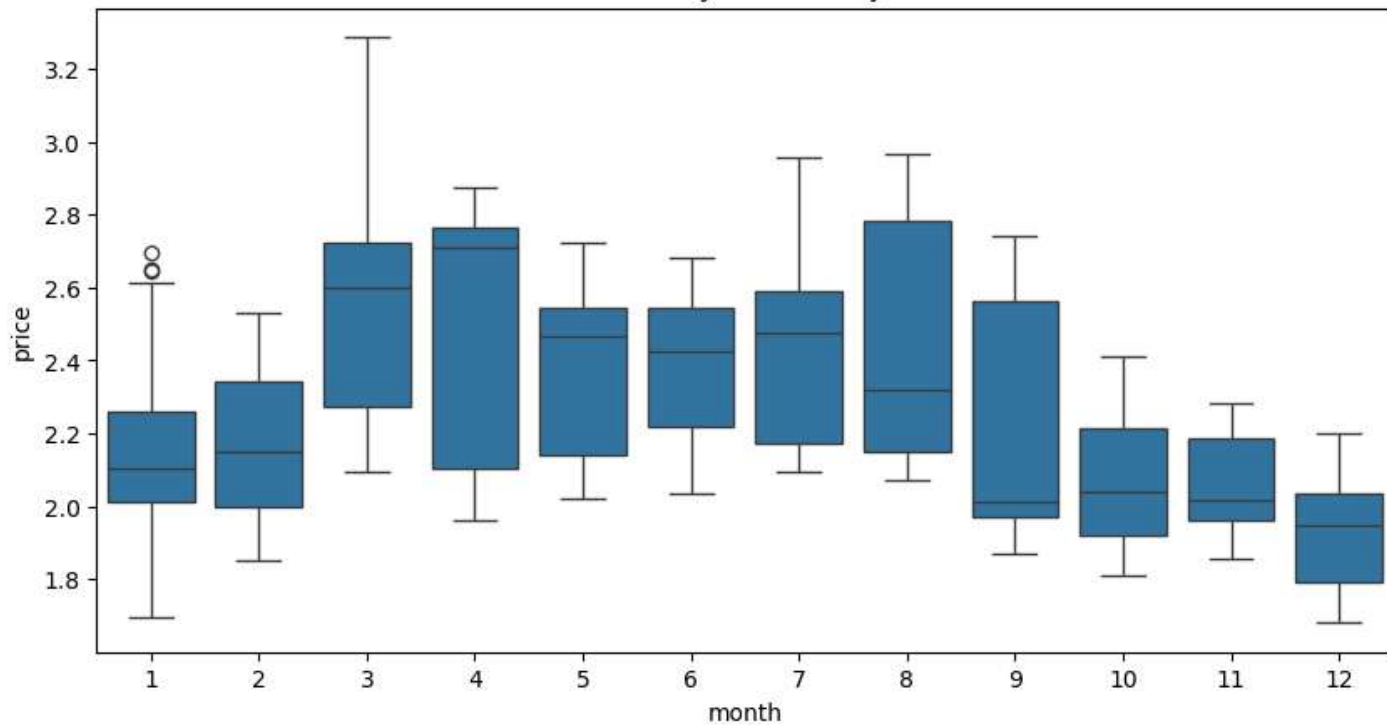
Oil vs Gas Price Relationship

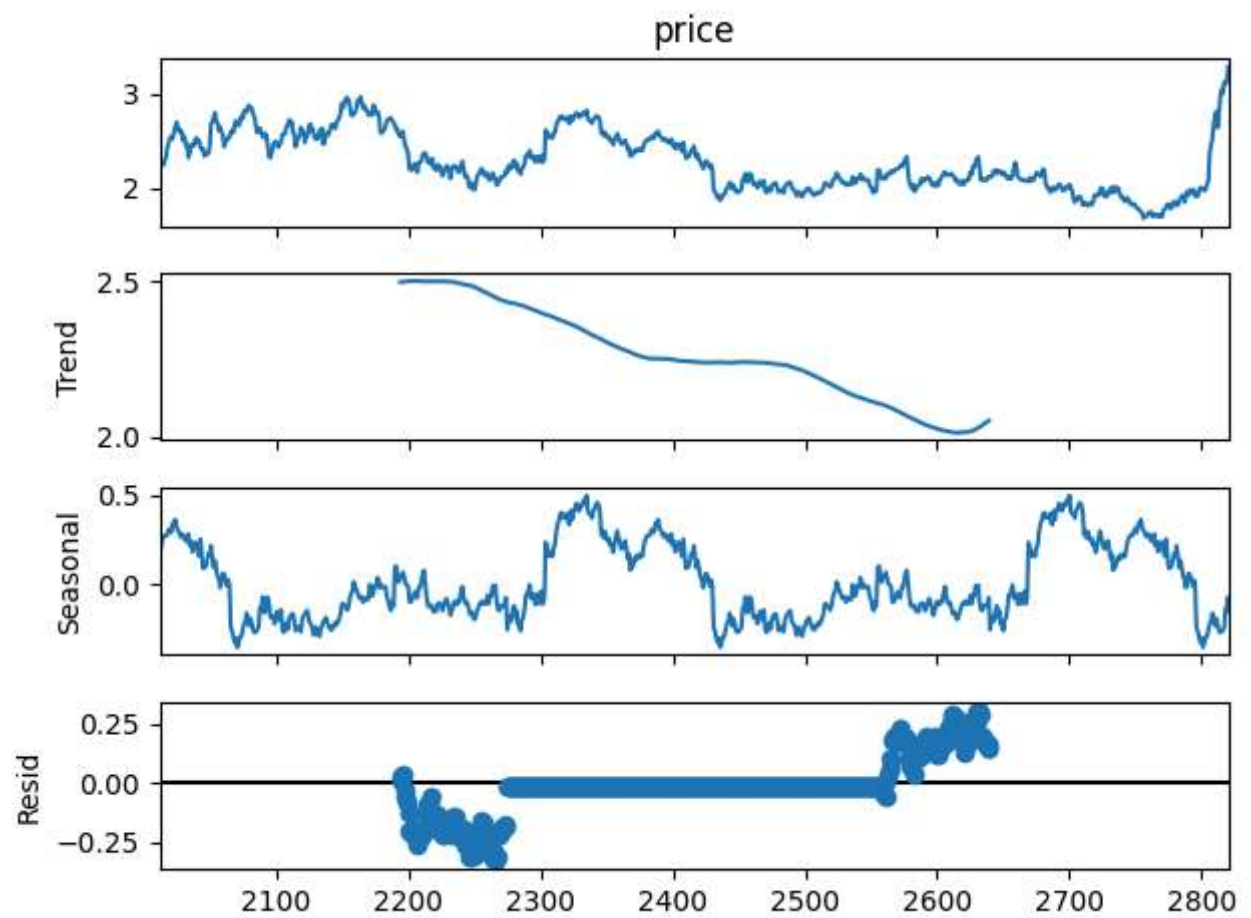
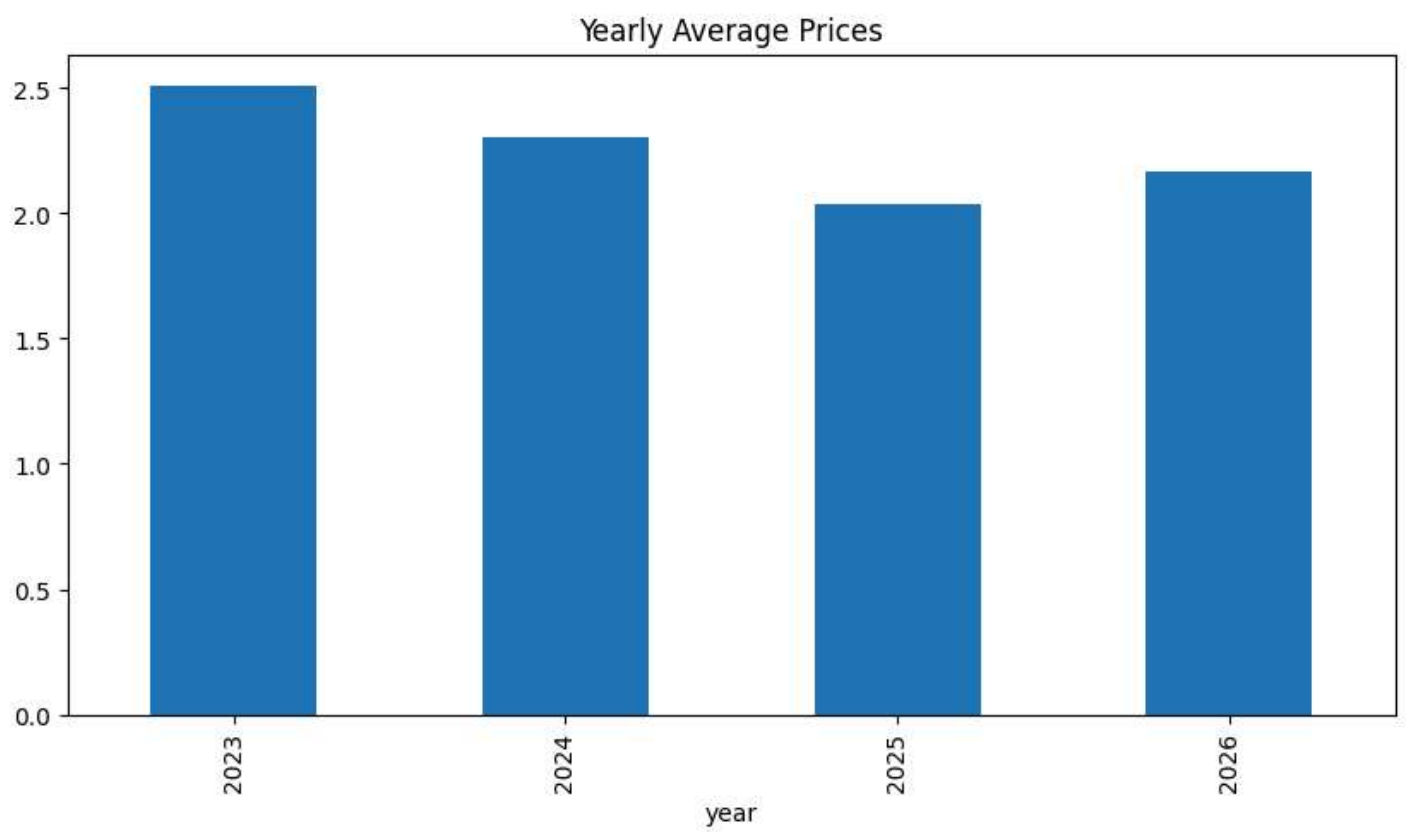


Oil (7 day lag) vs Gas



Monthly Seasonality



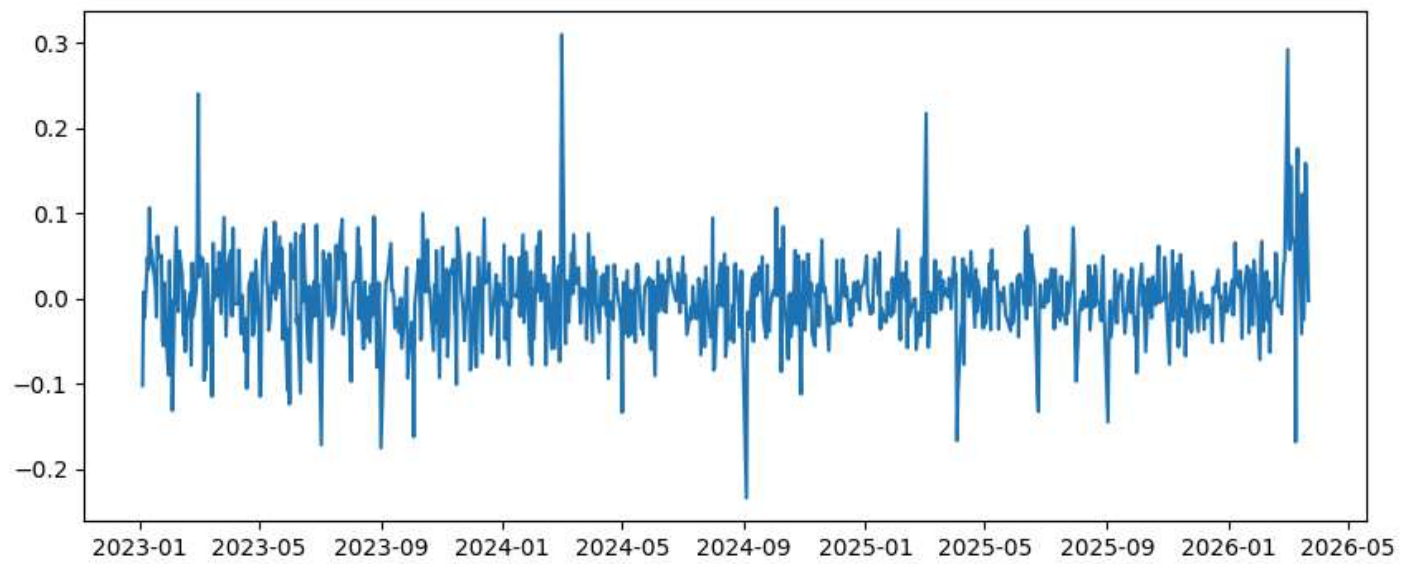


ADF Statistic: -1.3119889967516216

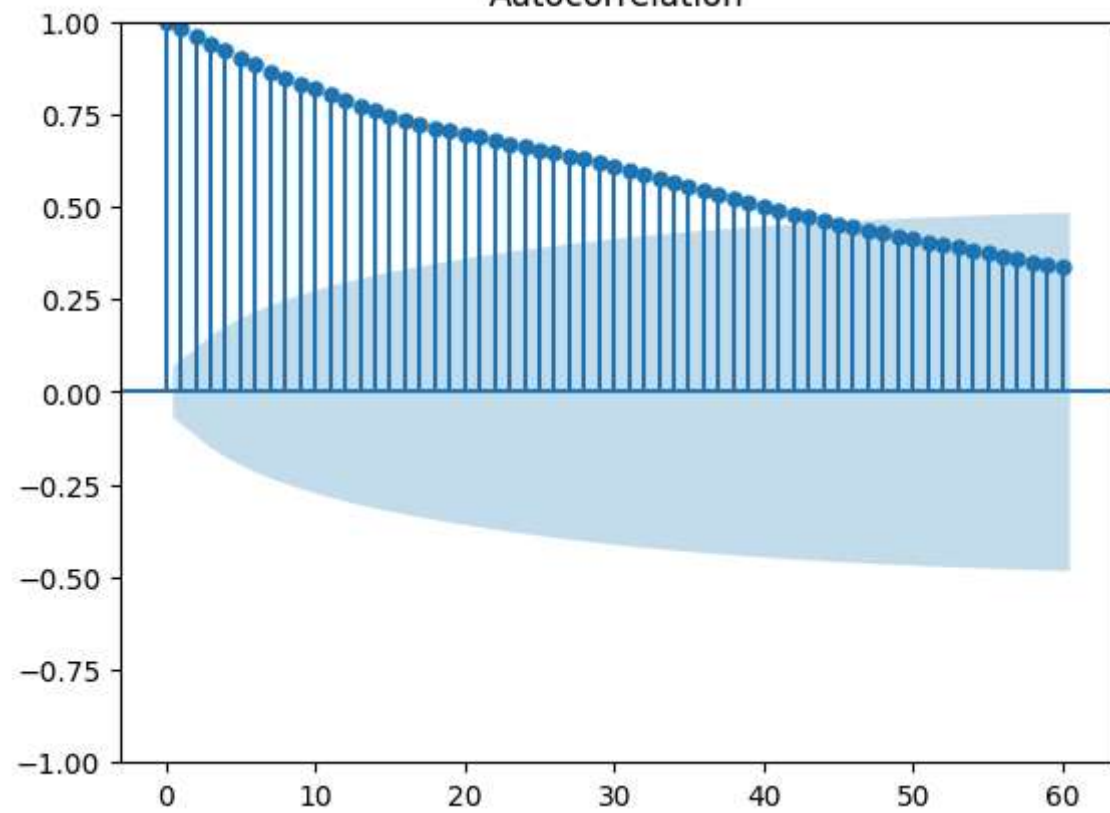
p-value: 0.6236696399371089

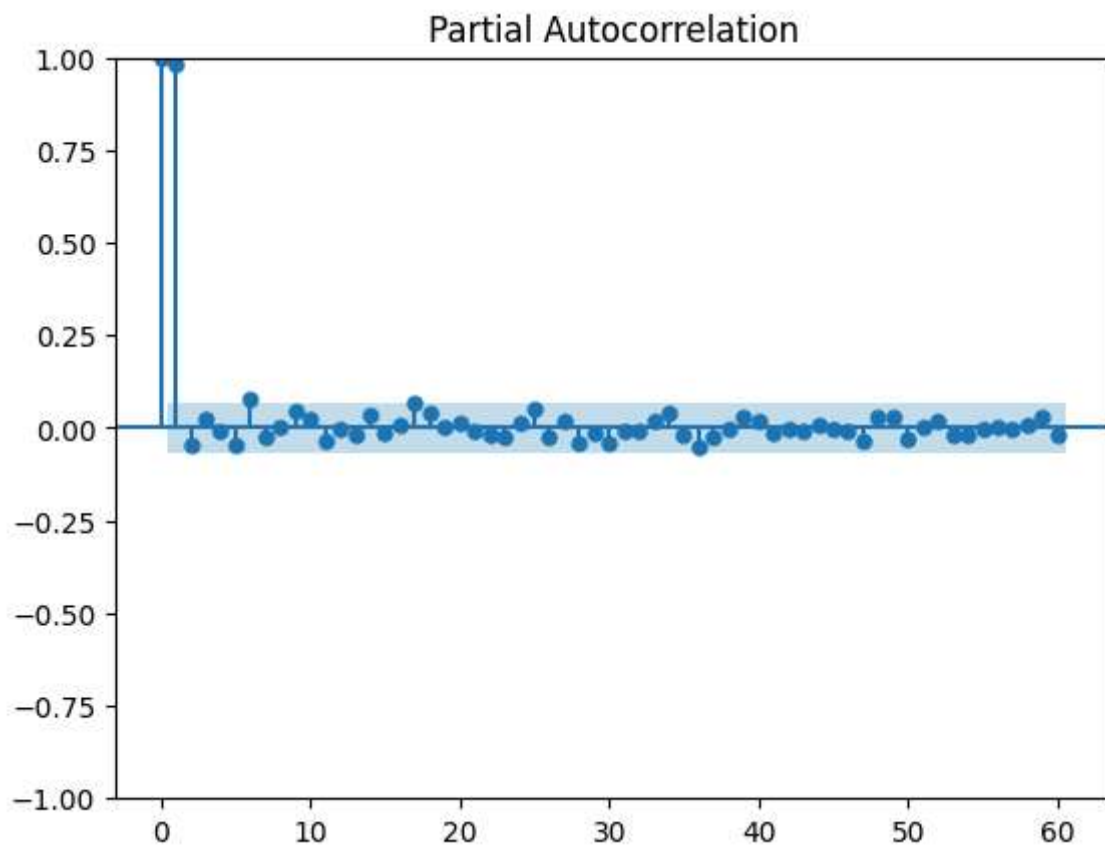
Series is NOT stationary

Differenced Series



Autocorrelation





Outliers detected: 2

===== EDA BUSINESS INSIGHTS =====

Average gas price: 2.28

Max price: 3.29

Min price: 1.68

Oil correlation: 0.8

EDA completed

Training ARIMA model

c:\Users\Aditi\projects\.venv\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:966: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.

warn('Non-stationary starting autoregressive parameters')

c:\Users\Aditi\projects\.venv\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.

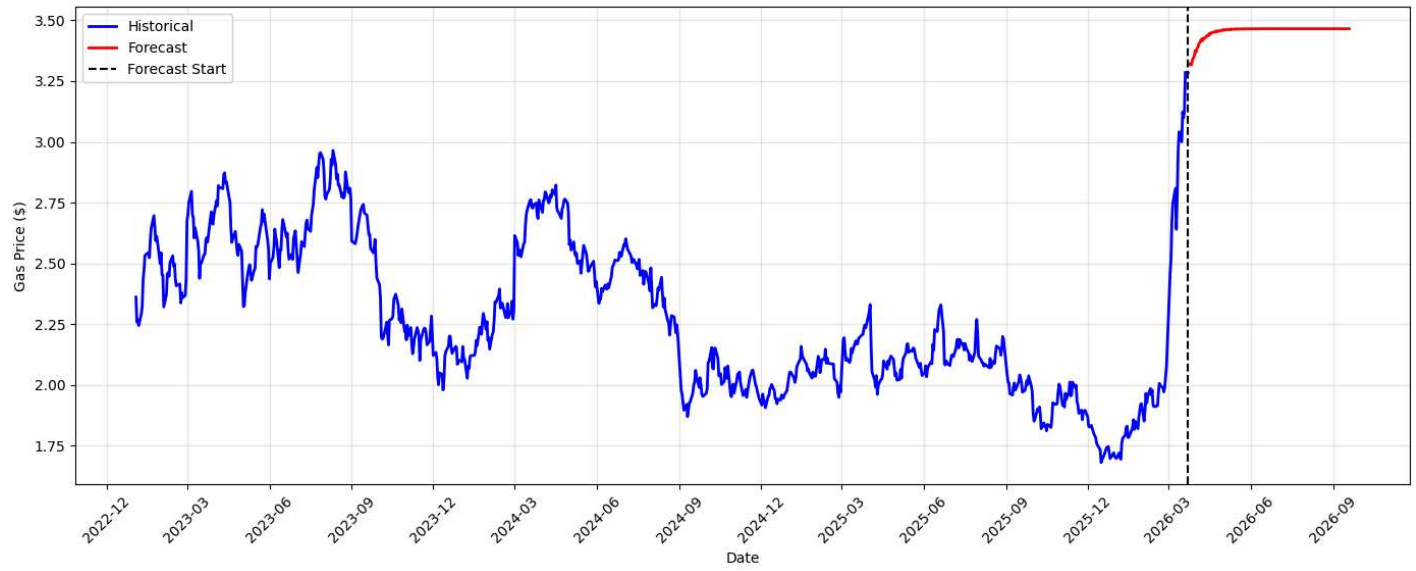
warn('Non-invertible starting MA parameters found.')

c:\Users\Aditi\projects\.venv\Lib\site-packages\statsmodels\base\model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals

warnings.warn("Maximum Likelihood optimization failed to "

Forecasting

Gas Price Forecast into 2026



===== FORECAST SUMMARY =====

Forecast start: 2026-03-24 00:00:00

Forecast end: 2026-09-19 00:00:00

Average forecast price: 3.45

Max forecast price: 3.47

Min forecast price: 3.32

===== SEATTLE MARKET ANALYSIS =====

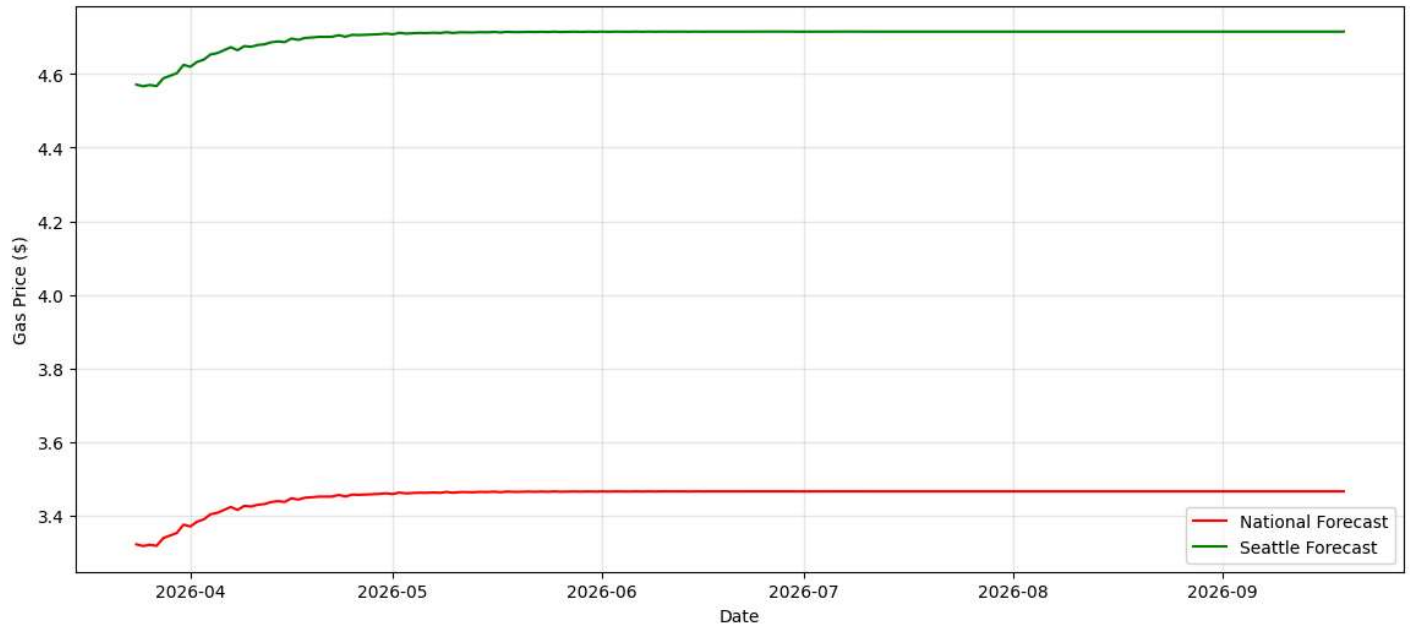
Applied Seattle regional premium: 1.25

Seattle forecast avg price: 4.7

Seattle max forecast: 4.72

Seattle min forecast: 4.57

Seattle vs National Gas Price Forecast



Index(['date', 'forecast_price', 'seattle_price'], dtype='str')

===== AI FORECAST ANALYSIS =====

Historical average price: 2.28

Forecast average price: 3.45

Forecast max price: 3.47

Forecast min price: 3.32

Oil correlation: 0.8

AI Insight: Prices show upward trend into 2026

AI Insight: Oil prices strongly influence gas prices

Forecast volatility: 0.03

===== AI SEATTLE MARKET ANALYSIS =====

Seattle average forecast: 4.7

Seattle max: 4.72

Seattle min: 4.57

Seattle regional premium: 1.25

Seattle AI Insight:

Seattle prices expected to remain elevated due to regional taxes and supply constraints.

Local prices projected to remain about 1.25 above national averages.

AI Risk Insight: Seattle prices may remain in high-cost range through 2026.

===== AI BUSINESS SUMMARY =====

National outlook: Moderate stabilization expected.

Seattle outlook: Continued price premium due to structural market factors.

Key drivers: Oil prices, regional taxes, supply logistics.

Model limitation: ARIMA may underestimate sudden price shocks.

EXECUTIVE SUMMARY:

Gas prices are expected to stabilize nationally while Seattle prices remain elevated due to structural regional costs.

Project completed